

Servidor Proxy SOCKS5 con Protocolo de Monitoreo

Keoni Dubovitsky Franco Ferrari Nicolas Mazzitelli Agustin Brunero

Protocolos de Comunicación, 2026C1

Resumen

Se desarrolló un servidor proxy SOCKS5 en C11 con I/O no bloqueante multiplexado, autenticación usuario/contraseña, conexión TCP saliente a IPv4, IPv6 y FQDN, resolución DNS fuera del hilo del selector, métricas, registro de accesos y un protocolo propio de monitoreo/configuración con cliente de terminal. El informe explica las decisiones de diseño, los problemas encontrados y los límites que surgen de la arquitectura y de las mediciones realizadas.

Índice

1. Descripción de protocolos y aplicaciones	2
1.1. SOCKS5	2
1.2. Protocolo de monitoreo	2
2. Problemas encontrados	2
3. Limitaciones de la aplicación	3
4. Posibles extensiones	3
5. Conclusiones	3
6. Ejemplos de prueba	3
6.1. Corrida Pampero/Linux	4
6.2. Stress local	4
7. Guía de instalación	5
8. Instrucciones para la configuración	6
9. Ejemplos de configuración y monitoreo	7
10. Documento de diseño del proyecto	7
10.1. Estados SOCKS5	7
10.2. DNS y lifetime	7
10.3. PMC	7

1. Descripción de protocolos y aplicaciones

La aplicación servidor actúa como intermediario entre clientes SOCKS5 y servicios TCP. En el mismo proceso ofrece un plano de administración separado, al que accede la aplicación cliente de terminal. Esta división evita que las operaciones administrativas se confundan con el tráfico que retransmite el proxy.

1.1. SOCKS5

El proxy acepta autenticación usuario/contraseña y el comando `CONNECT` para destinos IPv4, IPv6 o FQDN. Después de establecer la conexión saliente, retransmite bytes en ambos sentidos sin interpretar el protocolo de aplicación. Si un nombre resuelve a varias direcciones, conserva la lista y prueba la siguiente cuando una conexión falla. La decisión importante no fue reproducir los pasos de RFC1928 y RFC1929, sino integrarlos con lecturas parciales y con una máquina de estados que nunca presupone cómo se segmentan los datos en TCP.

1.2. Protocolo de monitoreo

El Protocolo de Monitoreo y Configuración (PMC) usa una conexión TCP persistente. Es textual y orientado a líneas terminadas en `CRLF`. Cada pedido empieza con un nombre de comando hasta el primer espacio o fin de línea; los campos restantes se separan por un único espacio. Para cada pedido el servidor devuelve una respuesta que comienza con `+OK` o `-ERR`. Antes de operar, el cliente negocia la versión y autentica una credencial administrativa.

Los comandos implementados son `ADD-USER`, `DEL-USER`, `LIST-USERS`, `METRICS`, `GET-CONFIG`, `SET-CONFIG` y `QUIT`. Las respuestas multi-línea usan prefijo de cantidad para delimitar la respuesta sin depender del cierre de la conexión. La gramática, los estados y cada respuesta posible se especifican en `docs/mgmt-protocol-rfc.md`.

El cliente de terminal oculta la negociación y el formato de cable. Traduce subcomandos como `metrics` o `add-user` a pedidos PMC y presenta resultados etiquetados, listas y mensajes de error, en lugar de copiar las líneas recibidas del socket.

2. Problemas encontrados

Problema	Resolución
Portabilidad Linux/macOS I/O parcial y pipelining	Se usaron macros de plataforma en el Makefile para exponer APIs POSIX bajo C11 sin romper macOS. Los parsers consumen desde buffers y preservan bytes sobrantes para el siguiente estado. Esto evita perder AUTH o REQUEST enviados en el mismo segmento.
Errores del selector base	Se corrigieron casos de flags no bloqueantes, límites de fd y headers no autocontenidos.
DNS bloqueante	La resolución se aisló en hilos <code>pthread</code> dedicados a <code>getaddrinfo</code> ; el resultado vuelve al selector por notificación.
Reuso de fd durante DNS	Las notificaciones de DNS llevan identidad del objeto de conexión para no despachar un resultado viejo sobre una conexión nueva.
Half-close pre-COPY	Si el cliente envía payload temprano y hace half-close mientras el <code>connect</code> está pendiente, el proxy conserva el payload y lo drena al origen al entrar en COPY.

3. Limitaciones de la aplicación

El selector actual usa `pselect` y queda limitado por `FD_SETSIZE`. Una conexión SOCKS5 establecida consume dos file descriptors principales: cliente-proxy y proxy-origen. Por eso el techo real debe medirse en el entorno de entrega, no inferirse sólo desde el código.

Las métricas se conservan sólo mientras vive el proceso. El access-log se escribe de forma síncrona con `fprintf/fflush`; esto facilita inspeccionar cada registro, pero puede introducir latencia cuando el dispositivo de salida es lento.

El apagado controlado tiene una decisión explícita: una segunda señal corta el drenaje graceful de conexiones activas, pero no destruye el selector ni el pool mientras haya resoluciones DNS pendientes. Esto evita use-after-free, al costo de esperar si `getaddrinfo` tarda.

El PMC autentica al administrador pero no cifra el canal. Debe utilizarse dentro de una red administrativa controlada o acompañado por una capa de transporte segura.

4. Posibles extensiones

- migrar el selector a `poll`, `epoll` o `kqueue` para superar `FD_SETSIZE`;
- persistir métricas y rotar logs;
- agregar TLS o un esquema de autenticación más fuerte al PMC;
- exponer timeouts y más parámetros de configuración en runtime;
- exportar métricas en un formato apto para monitoreo externo.

5. Conclusiones

El principal aprendizaje fue que I/O no bloqueante no significa reemplazar una llamada bloqueante por otra con una bandera. Cada evento habilita sólo una parte del trabajo: una lectura puede traer medio mensaje, una escritura puede aceptar algunos bytes y un `connect` puede quedar pendiente. Modelar ese progreso en estados y buffers hizo posible conservar datos y aplicar contrapresión sin detener a las demás conexiones.

La resolución de nombres mostró otro aspecto menos evidente: sacar `getaddrinfo` del hilo principal resuelve el bloqueo, pero introduce un problema de identidad y tiempo de vida. Tuvimos que razonar qué ocurre si el cliente se desconecta, el descriptor se reutiliza o el proceso comienza a apagarse mientras la resolución continúa. Aprendimos que la concurrencia debe analizarse por propiedad de los datos y no sólo por la cantidad de hilos.

Diseñar PMC también cambió nuestra forma de describir un protocolo. Los ejemplos sirven para ilustrar, pero no reemplazan una gramática, una máquina de estados ni reglas que delimiten cada respuesta. El prefijo de cantidad en respuestas multilínea y el orden en pipelining surgieron de preguntarnos cómo podría un cliente independiente interpretar el flujo sin conocer nuestra implementación.

Por último, las mediciones de carga nos enseñaron a separar una observación de una afirmación de capacidad. Las gráficas permiten comparar throughput, latencia y errores bajo las combinaciones ejecutadas; no demuestran por sí solas un máximo universal. El límite de `pselect`, dado por `FD_SETSIZE`, y el consumo de dos descriptors por conexión explican por qué el entorno y la metodología forman parte del resultado.

6. Ejemplos de prueba

Comando	Estado	Observación
---------	--------	-------------

<code>make clean && make</code>	Verificado local y Pampero	Compila <code>bin/server</code> y <code>bin/client</code> sin warnings visibles.
<code>make test</code>	Verificado local y Pampero	Unitarios C del parser, usuarios, métricas, logger, selector bloqueante y relay.
<code>make check PORT=12080</code>	Verificado local y Pampero	Flujos completos de negociación, autenticación, conexión, relay, DNS, métricas y PMC.
<code>make valgrind PORT=...</code>	Verificado en Pampero	Valgrind con tráfico: 0 errores, 0 leaks definitivos/indirectos/posibles.
<code>strace</code> sobre relay HTTP	Verificado en Pampero	En ambos sentidos se observó <code>recvfrom</code> → <code>sendto</code> sin un <code>pselect6</code> intermedio.
<code>test/stress/run_stress.py</code>	Corrida local acotada	Genera CSV y figuras reproducibles para las combinaciones elegidas.

6.1. Corrida Pampero/Linux

La corrida remota se ejecutó en `pampero.it.itba.edu.ar`, Linux 7.0.9-arch1-1, GCC 16.1.1 20260625. El build `make clean && make` terminó con BUILD OK.

Los unitarios reportaron todos los casos OK y cubrieron:

- negociación, autenticación y parseo de pedidos fragmentados;
- conexión, relay bidireccional, cierres parciales y contrapresión;
- notificaciones del selector, métricas y registro de accesos.

Las integraciones terminaron sin fallas y ejercitaron los mismos comportamientos sobre sockets reales. El único caso no ejecutado requería que el entorno resolviera un dominio a varias direcciones para forzar el reintento; esa precondition no estaba disponible en la corrida.

Valgrind con tráfico real reportó:

- FILE DESCRIPTORS: 2 open (2 inherited) at exit;
- definitely lost: 0 bytes, indirectly lost: 0 bytes y possibly lost: 0 bytes;
- ERROR SUMMARY: 0 errors from 0 contexts.

Los 5,658 bytes still reachable se consideran alcanzables del entorno/libc, no leaks definitivos del programa.

6.2. Stress local

El banco de stress se genera con `test/stress/run_stress.py`. El script levanta un origen controlado, ejecuta el servidor SOCKS5, consulta métricas por PMC y escribe CSV bajo un directorio de salida. Los parámetros principales son:

- concurrencia: cantidad de conexiones simultáneas por caso;
- payload: bytes transferidos por conexión;
- modo de destino: `ipv4` o `fqdn`;
- métricas recolectadas: conexiones OK/fallidas, bytes, wall time, throughput, tasa de conexiones, latencias p50/p95, REP no exitosos y deltas de métricas PMC.

La evidencia versionada corresponde a una corrida local acotada en macOS, guardada en:

`docs/stress/2026-07-05_local-smoke_v1/`

La corrida usó concurrencias 5 y 20, payloads 1024 y 8192 bytes, destinos IPv4 y FQDN, y una repetición por combinación. También hubo una prueba local adicional de 500 conexiones

con `conn_ok=500` y `conn_failed=0`. No se presenta como máximo absoluto del servidor ni como barrido completo en Pampero.

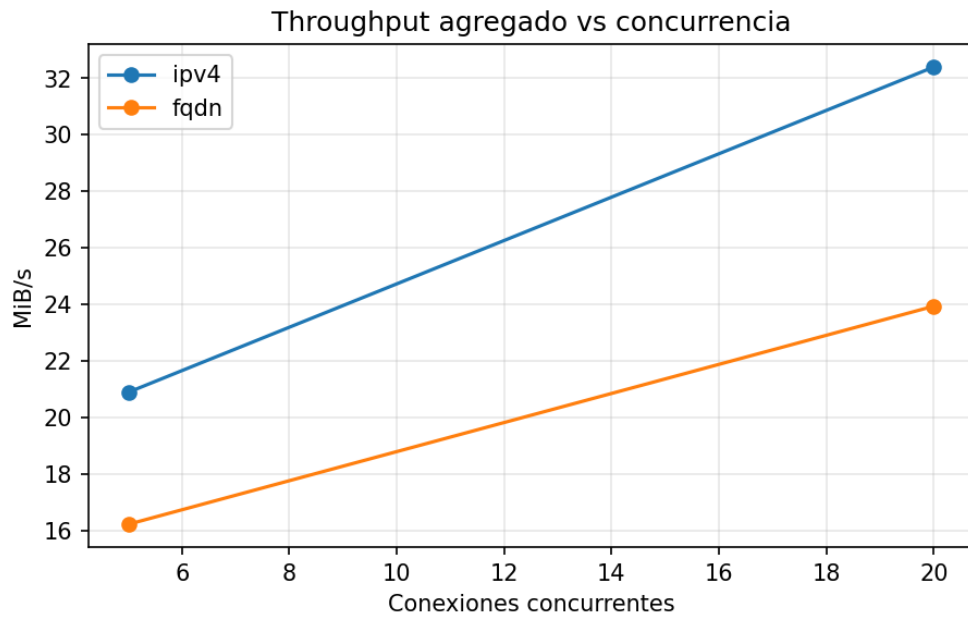


Figura 1: Throughput agregado de la corrida local por concurrencia y modo de destino.

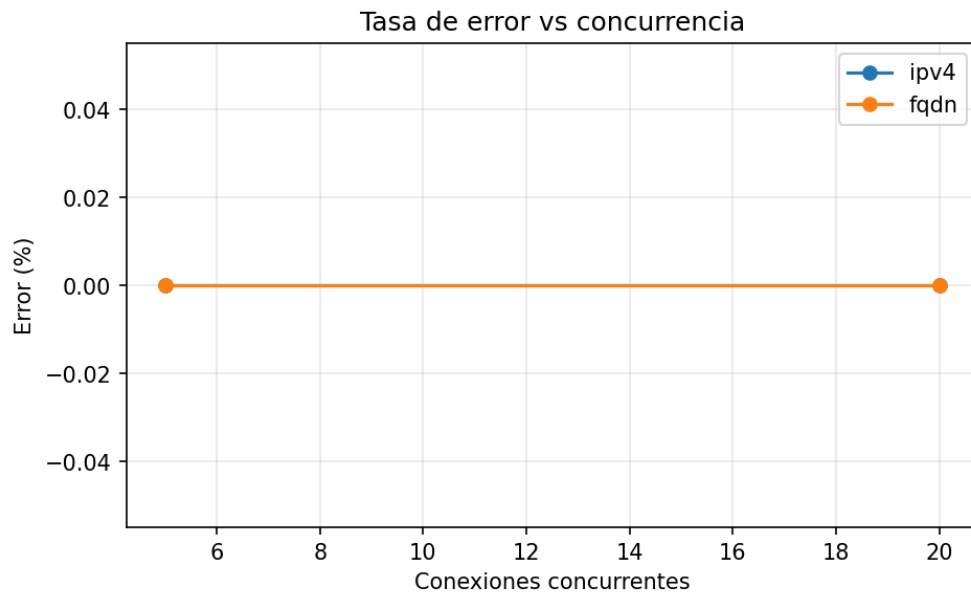


Figura 2: Tasa de error de la corrida local. No hubo conexiones fallidas.

7. Guía de instalación

Dependencias:

- compilador C11 (`gcc` o `clang`);
- `make`;
- `python3` para integraciones y `stress`;
- `valgrind` en Linux/Pampero para memoria/fds;

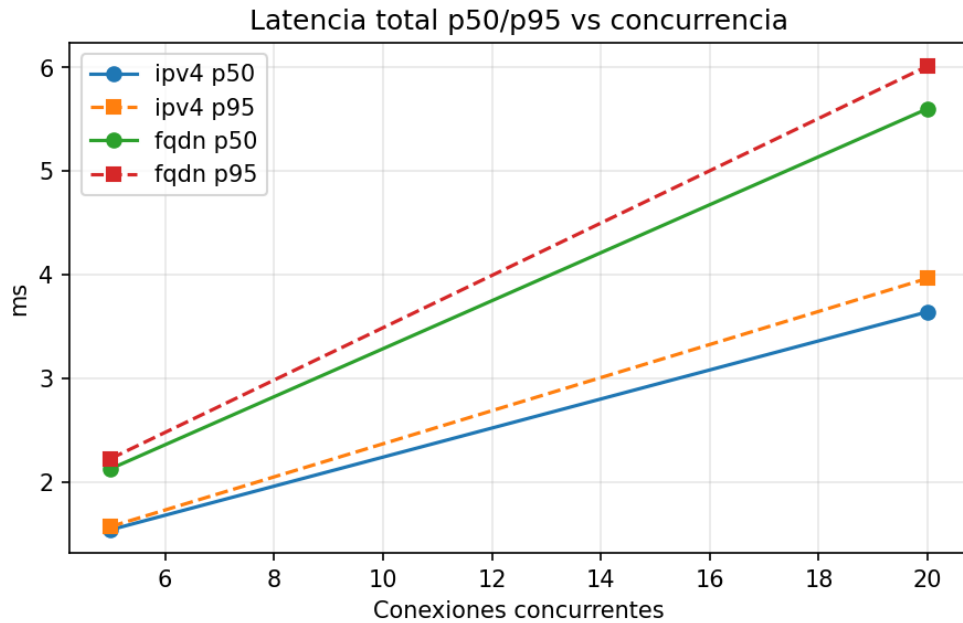


Figura 3: Latencias p50/p95 totales de la corrida local.

- matplotlib si se quieren regenerar gráficos de stress;
- rsync y ssh para el runner de Pampero.

Compilación:

```
make clean && make
```

Artefactos generados:

- bin/server: servidor SOCKS5 + PMC;
- bin/client: cliente CLI del PMC;
- docs/report/main.pdf: este informe.

Verificación remota recomendada:

```
PAMPERO_USER=<usuario> bash scripts/run-on-pampero.sh 12080
```

El script sube el repo a Pampero, recompila desde cero y ejecuta unitarios, integraciones sobre sockets y Valgrind con tráfico real.

8. Instrucciones para la configuración

Opciones principales del servidor:

```
./bin/server [OPCIONES]
-l <addr>          direccion SOCKS, default 0.0.0.0
-p <port>          puerto SOCKS, default 1080
-L <addr>          direccion PMC, default 127.0.0.1
-P <port>          puerto PMC, default 8080
-u <name>:<pass>   usuario SOCKS5
--admin u:p        credencial de administracion PMC
-N                desactiva disectores
-h / -v           ayuda / version
```

El tamaño de buffer se puede consultar y cambiar para conexiones nuevas mediante el PMC con GET-CONFIG buffer-size y SET-CONFIG buffer-size N.

9. Ejemplos de configuración y monitoreo

Ejemplos con el cliente:

```
./bin/client --admin admin:s3cr3t metrics
./bin/client --admin admin:s3cr3t add-user pablito pass1234
./bin/client --admin admin:s3cr3t list-users
./bin/client --admin admin:s3cr3t get-config buffer-size
./bin/client --admin admin:s3cr3t set-config buffer-size 32768
./bin/client --admin admin:s3cr3t del-user pablito
```

El cliente presenta los datos como operaciones de terminal, por ejemplo:

```
$ ./bin/client --admin admin:s3cr3t metrics
Metricas:
Conexiones historicas: 10
Conexiones concurrentes: 0
Bytes transferidos: 123456
Usuarios configurados: 2
Conexiones fallidas: 1
```

10. Documento de diseño del proyecto

El servidor usa tres piezas centrales del toolkit: **selector**, **stm** y **buffer**. El selector concentra los eventos de sockets; la máquina de estados separa cada fase del protocolo; los buffers resuelven lecturas y escrituras parciales.

10.1. Estados SOCKS5

```
HELLO ->AUTH ->REQUEST ->RESOLV? ->CONNECTING ->REQUEST_WRITE ->COPY ->DONE
```

COPY mantiene dos direcciones de relay: cliente a origen y origen a cliente. Cada dirección tiene su estado de lectura/escritura; al recibir EOF en un sentido se drena lo pendiente y se propaga **shutdown(SHUT_WR)** al par. Cuando una lectura produce datos, el relay realiza inmediatamente un único intento de escritura no bloqueante sobre el socket opuesto. En el caso favorable esto evita una vuelta adicional **pselect** → **write**; ante una escritura parcial o **EAGAIN**, conserva los bytes pendientes en el buffer y delega el drenaje posterior al selector. El límite de un intento por activación preserva la equidad del event loop entre conexiones concurrentes.

10.2. DNS y lifetime

La resolución FQDN corre en un hilo detached que sólo ejecuta **getaddrinfo** y notifica al selector. La notificación lleva identidad del objeto de conexión y cleanup asociado. El proceso de apagado espera a que no haya resoluciones DNS en vuelo antes de destruir selector y pool.

10.3. PMC

El PMC tiene su propio listener y su propia STM, pero comparte el mismo selector del servidor. En el cliente, la capa de protocolo resuelve conexión, framing, negociación y lectura de respuestas. La capa de terminal construye cada pedido e interpreta el resultado para mostrar listas, métricas, confirmaciones o errores. El cliente usa I/O bloqueante porque atiende una sola operación interactiva por proceso; esa elección no afecta la concurrencia del servidor.